

CACTUS: Auditable Evidence Admission and Failure-Aware Evaluation for Repository-Level Software Engineering Agents

Zhao Daping^a, Shi Honglei^{b,*}

^a*Winning Health Technology Group Co., Ltd., No. 9, Lane 99, Shouyang Road, Jing'an District, Shanghai, 200072, China*

^b*Taiyuan University of Technology, Yingxi Campus, No. 79, Yingze West Street, Wanbailin District, Taiyuan, 030024, Shanxi, China*

Abstract

Repository-level software engineering evaluation produces predictions, evaluator reports, source revisions, contracts, and provenance records that can disagree even when tests complete normally. We present CACTUS, an auditable evidence-admission architecture combining dual-lens outcome accounting, semantic binding checks, and a provenance-bound publication ledger. An audit of 71 archived attempts identifies 32 driver/source binding conflicts and admits 24 attempts. Two evidence-chain cases show how valid evaluator reports coexist with source conflicts. A controlled study applies 8 checksum-consistent mutation operators to 20 admitted seeds. All 160 mutations pass integrity checks and are rejected by the shared production semantic core; all 20 intact controls are admitted. A secondary archive study measures prediction-output behavior. The ledger carries admission decisions into manuscript values, tables, figures, and release checks. Within the audited archive, CACTUS detects the tested binding violations and preserves their provenance through publication. We do not use these data to estimate benchmark-wide repair effectiveness.

Keywords: software engineering agents, evidence admission, failure characterization, context selection, reproducibility

*Corresponding author.

Email addresses: zdp@winning.com.cn (Zhao Daping),
shihonglei0042@link.tyut.edu.cn (Shi Honglei)

1. Introduction

Software-engineering agents translate an issue into a patch against a particular repository revision (Jimenez et al., 2023; Yang et al., 2024; Xia et al., 2024). An agent needs the relevant definitions and dependencies, but a repository can exceed its available context budget. Retrieval methods such as ReACC and RepoCoder address context acquisition (Lu et al., 2022; Zhang et al., 2023). Supplying more context does not guarantee that a model uses it consistently (Liu et al., 2023a). Context selection therefore needs an explicit budget and an inspectable selection rule.

Repair assessment has a separate evidentiary requirement. A prediction can be rejected before tests run. A runner can exit successfully while its evaluator reports an empty patch or an infrastructure failure. A valid evaluator report can disagree with the source identity declared by the experiment contract. An aggregate progress flag does not resolve these distinctions. Reporting repair outcomes requires retaining the prediction, evaluator report, source revision, contract, and billing record as separate evidence objects.

CACTUS makes these evidence distinctions explicit. Its Failure-Aware Context Optimization (FACO) selector exposes budget and selection decisions for inspection; diagnosticity is a design objective rather than an established outcome. Eligible source syntax sites constrain edit locations, making patch generation failures detectable. The Dual-Lens protocol records evaluator validity independently of repair outcome. The admission gate then checks whether the complete evidence chain—prediction, evaluator report, source revision, contract, and billing record—is mutually consistent.

CACTUS records prediction, evaluator, source, contract, authorization, and billing evidence separately, then admits an observation only when the corresponding bindings agree. A layered failure model distinguishes repair mechanisms from failures of evaluation and evidence integrity. We audit repeated attempts in a retry archive. A separate prediction archive supports output-format analysis; it contains no official evaluator reports for assessing comparative repair performance.

This paper addresses the following research questions:

- **RQ1** (Failure characterization): How can repair failures and evidence failures be distinguished without mixing tasks and attempts?

- **RQ2** (Evidence admission): Which evidence-admission failures occur in the retained archive, and what do controlled bundle mutations establish about their detection?
- **RQ3** (Prediction-output behavior; secondary study): How does the incidence of nonempty, diff-marked prediction output vary across strategies in the retained matched archive?

Contributions.

1. A four-layer conceptual failure model that separates repair mechanisms from evaluation and evidence-integrity failures, with distinct task and attempt units.
2. A source-bound admission pipeline whose retained archive exposes driver/source binding conflicts; a valid evaluator report alone cannot resolve these conflicts.
3. Controlled checksum-consistent mutations that exercise the production admission core, with a secondary archive analysis of prediction-output behavior.
4. A unified value ledger and rendering gate binding manuscript values, tables, figures, and PDF release to retained inputs.

The architecture also retains inspectable context selection and source-owned editing. These components supply evidence objects to the admission pipeline.

2. Related Work

Research standards and scope of inference. The ACM SIGSOFT empirical standards describe method-specific expectations for research design and reporting (Ralph et al., 2020). Systematic-mapping guidelines distinguish constructing a classification scheme from validating and reporting it (Petersen et al., 2015). These standards motivate making evidence units and limitations explicit. Our retained-archive audit is not a systematic mapping study, and its conceptual failure model is not an independently coded empirical taxonomy.

Repository-level agents and context acquisition. Surveys of LLMs for software engineering and code model benchmarks place repository repair within a wider set of generation, testing, and maintenance tasks (Hou et al., 2024; Zheng et al., 2023). Their breadth reinforces the need to specify the evaluated

task and endpoint. SWE-bench moves code evaluation from standalone function generation toward issue-based repository repair (Jimenez et al., 2023; Chen et al., 2021). SWE-agent studies the agent-computer interface, Agentless separates localization, repair, and validation, and OpenHands provides an execution platform (Yang et al., 2024; Xia et al., 2024; Wang et al., 2024). AutoCodeRover uses structural search and CodePlan coordinates repository changes through planning (Zhang et al., 2024; Bairei et al., 2023). Retrieval-augmented generation, ReACC, and RepoCoder supply related foundations for context acquisition (Lewis et al., 2020; Lu et al., 2022; Zhang et al., 2023). Long-context position effects, ContextBench, and Agent Retrieval Bench motivate inspecting what an agent receives and uses (Liu et al., 2023a; Li et al., 2026; Qin and Xie, 2026). CACTUS retains context selection as an inspectable component, while its central question concerns admission of the resulting evaluation observation.

Benchmark construction and repair correctness. Defects4J, ManyBugs, and IntroClass package faults, revisions, and tests to support controlled repair studies (Just et al., 2014; Le Goues et al., 2015). Plausible patches can pass the available tests without implementing a correct repair (Qi et al., 2015; Smith et al., 2015); recent agent traceability work further studies repair behavior and validation (Ceka et al., 2026). Thus evidence admission does not replace patch-correctness assessment. SWE-bench Lite and Verified define task universes, while SWE-Bench Pro broadens the long-horizon setting (Jimenez et al., 2024; SWE-bench Team, 2024; Deng et al., 2025). SWE-rebench addresses fresh task collection and contamination; benchmark design for learning-based APR also requires attention to training-data exposure (Badertdinov et al., 2025; Renzullo et al., 2025). Rephrased benchmark samples can evade string-based decontamination, so retained task hashes alone cannot rule out training-data exposure (Yang et al., 2023). These concern task quality and generalization, rather than whether a specific retained observation agrees with its frozen contract.

Constrained generation and mutation-based validation. PAR derives repair templates from human-written patches, while syntax-guided synthesis combines a grammar with a logical correctness specification (Kim et al., 2013; Alur et al., 2013). CACTUS instead binds edits to retained source spans and checks local applicability; it neither learns PAR templates nor solves a synthesis specification. Mutation testing deliberately changes programs to

assess fault detection (Jia and Harman, 2011). Our controlled study transfers this idea to evidence objects: mutations challenge admission predicates, rather than the functional correctness of executable programs.

Evaluation infrastructure and measurement units. AgentBench and AgentBoard examine agent evaluation across interactive environments and expose richer process information than a single terminal score (Liu et al., 2023b; Ma et al., 2024). CACTUS specializes this separation to prediction, official evaluation, raw runner status, and publication admission, keeping tasks and repeated attempts distinct. Containerization alone does not guarantee a reproducible build (Malka et al., 2026); OCI content descriptors make content digests explicit, but a digest must still be bound to the intended artifact and task (Open Container Initiative, 2024). Our image-reference check verifies a retained declaration, rather than claiming a newly observed runtime identity.

Provenance, reproducible research, and artifacts. PROV-DM and PROV-O describe entities, activities, and derivations, while PROV constraints formalize consistency requirements (Moreau and Missier, 2013; Lebo et al., 2013; Cheney et al., 2013). CACTUS adds domain-specific bindings among predictions, evaluator reports, source revisions, contracts, authorization, and billing. FAIR principles address discoverability and reuse; reproducible computational research and scientific-computing practices emphasize retained inputs, workflows, and versioning (Wilkinson et al., 2016; Sandve et al., 2013; Wilson et al., 2017). Open-science work in software engineering and empirical analysis of LLM-for-SE reproducibility identify continuing artifact and environment gaps (OliveiraJr et al., 2024; Siddiq et al., 2025). Artifact-review policies distinguish availability, evaluation, and independently validated results (Association for Computing Machinery, 2020). We accordingly distinguish paper regeneration, admission verification, and full experimental replay.

Prior studies evaluate whether agents repair tasks, retrieve useful context, resist contamination, or execute reproducibly. CACTUS treats the individual evaluation observation as the unit of audit. Its prediction, evaluator, source, contract, and provenance records must agree before the observation enters a scientific result. The publication ledger preserves the admission decision as observations are transformed into manuscript values.

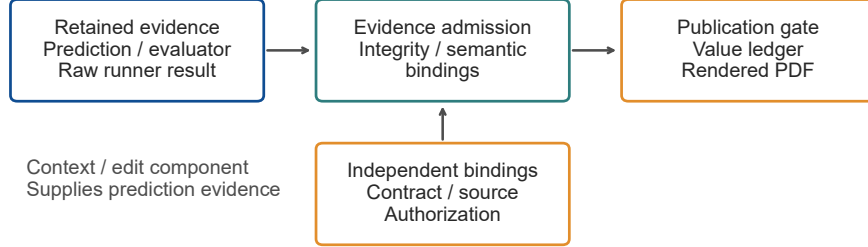


Figure 1: Evidence flow from retained prediction, evaluator, and raw objects through admission to the publication gate. Independent contract, source, and authorization bindings constrain admission; the context and edit component supplies prediction evidence.

3. Method

3.1. Evidence objects and admissibility

An evaluation observation comprises a task identity, prediction, evaluator report, raw runner result, source revision, frozen contract, authorization, and billing evidence. Each object has a different role: tests adjudicate the patch, source bindings identify the implementation, and provenance records establish how the observation was produced. File hashes protect retained contents; equality of hashes alone does not establish consistency between these roles.

Let I denote bundle integrity and S the conjunction of semantic binding predicates. Admission requires $I \wedge S$. An official evaluator result is recorded separately, so an evaluator-valid observation can still be evidence-inadmissible. Missing required evidence fails closed. The gate does not infer patch correctness from provenance, nor correct provenance from passing tests.

3.2. Dual-Lens outcome accounting

The algorithm lens distinguishes valid resolved and valid unresolved evaluations. The evaluator lens records rejected predictions, missing reports, infrastructure failures, and other unadjudicated attempts. A strict evaluator-valid observation requires a completed official task result, mutually consistent resolved/unresolved accounting, and absence of evaluator error, infrastructure, ambiguity, empty-patch, or unstopped-container evidence.



Figure 2: Dual-lens accounting separates evaluator validity from repair outcome. Publication admission additionally requires integrity, source, contract, and billing consistency.

Evidence admission is a further condition. It checks sealed bundle integrity, task identity, contract and authorization binding, source/driver identity, billing reconciliation, prediction evidence, and raw/evaluator agreement. Evaluator validity alone cannot establish these properties. A result can thus have a valid official evaluation while remaining excluded from publication.

3.3. Admission semantics

The production implementation resides in `cactus/governance/admission.py`. Both the publication source-admission importer and the controlled mutation harness invoke its `evaluate_admission` entry point. Evaluator and prediction predicates are also shared. The checker is read-only: it returns an admission decision, detection stage, reason codes, evaluator status, independently read source digests, and elapsed validation time.

Integrity checks validate checksums, configuration hashes, listed outputs, environment bindings, and raw/manifest identity before semantic checks execute. The semantic stage compares the bundle contract against its independently frozen counterpart and the authorization against the canonical contract digest. It checks the source commit and clean-tree assertion, then reads the driver and derivation bytes at that commit from retained Git history. A bundle’s own driver declaration cannot serve as proof of its source identity.

Task identity is checked across the manifest, resolved configuration, prediction, and evaluator report. Official completed/resolved/unresolved counters and identifiers must agree. The raw result must agree with the evaluator. A billing record must reconcile internally and, when usage is retained, agree

with the attempt’s raw token and spend observations; a self-consistent record from another attempt is insufficient. Retained model versions and task-specific evaluator image references are compared with the frozen contract. These checks establish declared evidence consistency, not an independently verified provider snapshot or a fresh runtime-image inspection.

3.4. *Publication ledger*

`paper/publication_sources.json` pins the evidence inputs and current implementation files. A deterministic generator computes the value ledger. Each value has a stable identifier, value type, source path, full hash, selector, and transformation. Types distinguish supporting repair observations, archive audit counts, implementation parameters, and protocol identifiers. The manuscript uses stable value references; unknown or blocked identifiers stop lint and compilation.

Tables are generated as references to these identifiers. Figure annotations and plotted values read the same ledger, and sidecars record the consumed value identifiers. The publication gate follows the manuscript input graph, verifies registered values and permitted sections, checks artifact and input hashes, and checks the generator binding. A separate source gate recomputes admission from the retained evidence. Merely matching a CSV hash is insufficient to pass that gate.

The same pipeline can regenerate publication artifacts without model calls or evaluator execution. This is a regeneration claim, not an end-to-end experimental replay claim. Original RUN bundles, contracts, source history, and billing evidence remain necessary for independent admission verification.

3.5. *Context-selection component*

A task supplies a repository revision, issue description, and evaluator specification. CACTUS selects source units under an explicit token budget, combining lexical, symbol, graph, test, and path relevance with coverage gain and redundancy penalties. It recomputes marginal utility after each selection, includes parent-closure cost, and resolves ties deterministically. Source hashes and selection telemetry bind the selected context to the revision. These are inspectable implementation properties; this study does not optimize the scoring weights.

Source-owned edit sites bind paths, spans, old text, and digests. The renderer rejects stale, overlapping, duplicate, and no-op edits, reparses modified files, and emits a unified diff. Supported dependent transactions retain

helper/caller obligations and require their validation closure before atomic emission. These mechanisms constrain local edit applicability. Their exact constants and supported semantic obligations are retained in Appendix A, while the secondary prediction-output analysis appears in Section 5.7.

3.6. Controlled semantic mutation design

Following the fault-seeding rationale of mutation testing (Jia and Harman, 2011), we define evidence-specific operators and check their intended detection stage. This adaptation does not establish the representativeness of all real evidence failures or a program-test mutation adequacy score. We select 20 admitted seed bundles in deterministic inventory order and copy them into a new validation directory. Each seed supplies an unmodified control and a case for each of 8 operators. Alternate predictions, billing records, source revisions, authorization records, and image references come from retained evidence for other tasks or contracts; no model or evaluator is rerun.

After mutation, the harness regenerates configuration hashes, prediction bindings, manifest output hashes, environment bindings, the local contract digest, and checksums in each copy. The independent contract, authorization, and source lookup remain external references. The study requires every copy to pass integrity, every control to be admitted, and each mutation to reach its specified semantic reason. The original seed tree hashes must remain unchanged. Substituting the driver digest tests equality with the independently frozen contract; source-revision substitution additionally exercises the driver/source rule. Substituting an evaluator image tests its task-specific contract identity, rather than executing a changed evaluator binary.

The oracle labels a mutation positive and an unmodified control negative; a detection is rejection. We report TP, FN, FP, TN, recall, specificity, false-positive rate, and stage per operator. Latency covers admission of materialized files in one warm process, including cached source reads; copying and resealing are excluded. The percentile latency uses the nearest-rank definition. The earlier integrity-oriented suite is retained as separate historical validation, not relabeled as this production-core execution.

AI-assisted implementation preparation. ChatGPT, OpenAI Codex, and Cursor AI-assisted features were used for coding assistance, testing, document typesetting, and compilation. OpenAI Codex (OpenAI; accessed 7–8 October 2026) assisted with implementation of statistical and validation scripts, publication tooling, and presentation-related code. The authors

reviewed the AI-assisted code and verified the reported numerical outputs against the fixed inputs and retained artifacts. Source changes and offline test outcomes are retained for inspection. These development activities did not constitute new model-based repair experiments or independent human review of the research claims.

4. Evidence Governance and Study Design

4.1. *Immutable historical evidence*

The audit reads every directory beneath `experiments/runs`, including incomplete and diagnostic directories. It checks checksum syntax, safe paths, listed files, manifest outputs, configuration identity, environment bindings, and raw/manifest identity. Full JSON Schema validation is performed independently of the legacy minimal validators. A missing manifest is an explicit integrity failure, not a passing exclusion.

An inventory records current file hashes and detected limitations. Creating this inventory does not certify that a damaged historical bundle was originally intact. The archive-integrity verdict remains failed when retained evidence is defective. A publication admission view cannot repair that verdict, rewrite old checksums, reconstruct missing measurements, or grant an owner exception.

4.2. *Historical snapshot withdrawal*

The older task snapshot contains 500 records. Its earlier synchronization procedure used progress records and could map a completed non-resolved execution to an unresolved task without strict evaluator validation. The snapshot also lacks a validated terminal-attempt selection chain required by the frozen final-outcome policy. Its currently admitted row count is 0.

The original CSV and associated RUNs remain available for historical investigation. Its aggregate resolutions, evaluator-valid denominator, and rates are not used as current effectiveness results. The paper also withholds historical token, cost, timing, repository, and temporal plots whose selected records have not passed complete admission. This withdrawal concerns evidence eligibility; it does not assert that every historical prediction or recorded success was false.

4.3. Failure-selected Lite retries

The later cohort comes from previously failed tasks in a deterministic local subset of SWE-bench Lite (Jimenez et al., 2024). Source revisions, validation logic, and contracts changed during development. All archived attempts remain visible, and contract groups are reported separately. A resolved observation under a later contract is not a matched comparison with an earlier failed attempt.

The retry source inventory is pinned by full hashes and checked against the retained RUNs, frozen contracts, source revisions, and admitted registry. Supporting records have partial reproducibility and descriptive scope. No attempt is relabeled as primary evidence. Exclusion reasons may overlap. Repeated attempts never become additional independent tasks, and selecting the best retry is not an estimator of single-attempt benchmark performance.

5. Results

5.1. Failure model and units of analysis (RQ1)

The conceptual model describes four layers: prediction/generation (A), execution/functional repair (B), evaluator/infrastructure (C), and evidence-binding/admission (D). Labels can overlap because a defective prediction and an inconsistent evidence chain may occur together. An observation records its layer, detection stage, and consequence for admissibility. We do not claim mutually exclusive or exhaustive empirical subtypes.

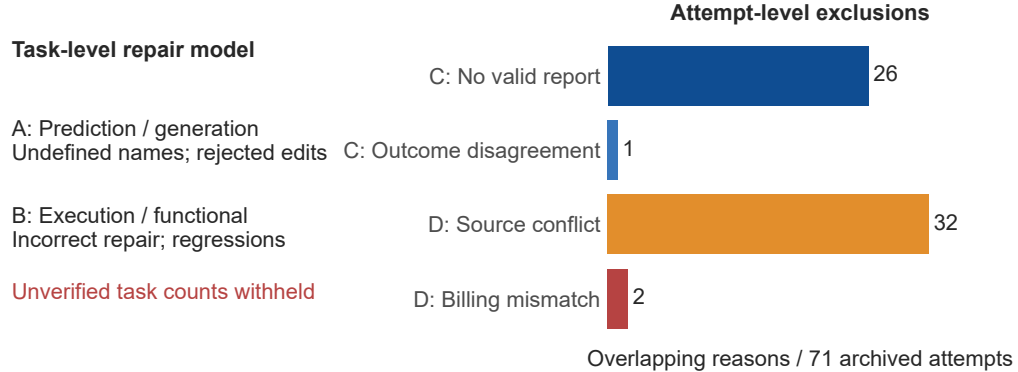


Figure 3: Layered failure model with explicit units. Left: task-level repair categories are conceptual; unverified task counts are withheld. Right: separate attempt-level exclusion counts in the retry archive. These reasons overlap and are not complete layer totals.

Task-level repair categories. Table 1 defines task-level categories without presenting unverified counts. The archived task-classification summary conflicts with its own listed task identities and repository assignments. It also lacks complete per-task annotations from which category intersections could be reconstructed. We preserve this source for investigation and withhold its aggregate success, failure, and repository-disparity counts. A complete task disposition requires an admitted terminal observation for every task; the present archive does not establish that disposition.

Table 1: Conceptual task-level repair failure model. Empirical counts are withheld pending a consistent task ledger and independent coding. Static detection tools are proposed candidates, not validated detectors in this study.

Layer		Mechanism	Detection boundary
A:	Prediction/generation	Undefined names, incompatible expressions, empty/no-op or rejected predictions	Renderer checks can reject unsupported edits. Undefined-name and type tools require a separate replay study.
B:	Execution/functional	Incorrect repair logic or regression after patch application	A completed, valid official evaluator report is required to classify repair outcome.

Attempt-level evidence failures. Table 2 reports only attempt-level counts recomputed from the retained retry inventory. Missing valid reports are an admission condition; they do not prove a particular infrastructure root cause. Likewise, source and billing conflicts concern admissibility rather than patch correctness. We report separate subtypes instead of treating a subtype count as the total for a layer.

5.2. Observed admission failures (RQ2)

The retry archive contains 71 attempts across 30 tasks. Bundle integrity checks pass for 71 attempts, while 45 have strictly valid official evaluator reports. Complete source and admission checks retain 24 attempts across 20 tasks. Table 4 reports these distinct units. Among admitted attempts, 3 were resolved, covering 3 distinct tasks: `astropy__astropy-6938`, `django__django-11001`, and `django__django-11049`. These are supporting repair observations within a

Table 2: Attempt-level evaluator and evidence exclusions among 71 archived attempts. Rows can overlap and must not be summed as distinct failures.

Layer		Evidence condition	Attempts
C:	Evalua-	No strictly valid official evaluator report	26
tor/infrastructure			
C:	Evalua-	Raw/evaluator outcome disagreement	1
tor/infrastructure			
D:	Evidence-	Contract driver hash differs from the bound	32
binding/admission		source driver	
D:	Evidence-	Billing record not reconciled	2
binding/admission			

Table 3: Denominator audit for admitted observations. Counts consume ledger references. Stable source identifiers are shown on separate lines to preserve table width.

Observation	Numerator	Denominator / unit	Ledger identifiers
Valid evaluation	45 attempts	71 archived at-tempts	<code>retry.evaluator_valid</code> <code>retry.archived</code>
Admission	24 attempts	71 archived at-tempts	<code>retry.admitted</code> <code>retry.archived</code>
Task coverage	20 tasks with admitted at-tempts	30 distinct retry tasks	<code>retry.admitted_tasks</code> <code>retry.tasks</code>
Admitted repair	3 resolved at-tempts	24 admitted at-tempts	<code>retry.resolved_runs</code> <code>retry.admitted</code>

failure-selected development cohort. We report neither a pooled benchmark rate nor a component improvement estimate from them.

5.3. Contract-binding conflicts exposed

The admission pipeline’s most consequential finding is the contract/source binding conflicts. Among 71 archived attempts, 32 declare a driver digest that differs from the driver at the bound source commit. These conflicts were identified by the retrospective source-admission audit. A preflight can check source identity before evaluation, but this retrospective observation does not measure prevented evaluator executions. The conflict concerns evidence identity rather than patch correctness.

Table 4: Archive audit and supporting observations: retry admission accounting. Rows distinguish attempts (individual executions) from distinct tasks (unique SWE-bench issues). Excluded attempts are retained in the archive inventory but excluded from publication. Separate attempt and task columns make the units explicit.

Stage	Attempts	Tasks
Archived	71	30
Bundle integrity	71	—
Official evaluator valid	45	—
Admitted	24	20
Resolved, admitted	3	3
Excluded	47	—

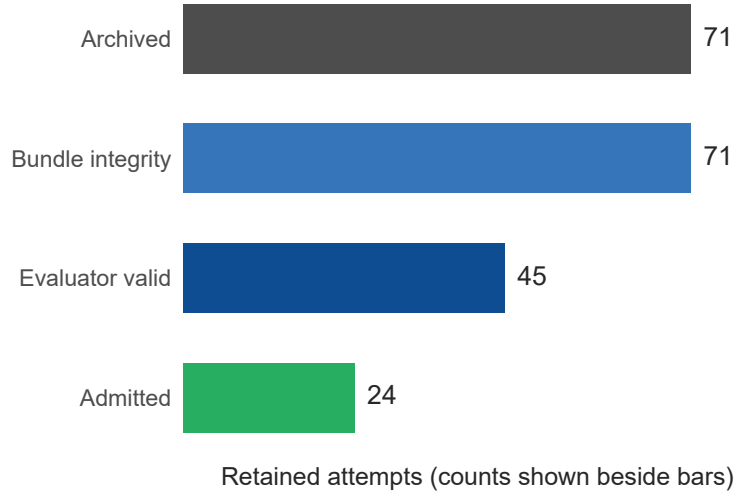


Figure 4: Archive audit: successive evidence conditions. Excluded attempts are retained in the inventory. Bar values and annotations consume the same value ledger as Table 4.

Table 5 preserves overlapping reasons. A valid evaluator report cannot clear this source/contract conflict. The original contract and RUN evidence were retained without retrospective correction. We do not know how many similar conflicts are present in conventional benchmark reports that lack an admission pipeline.

Table 5: Archive audit: overlapping exclusion reasons for the 71 retry attempts. A single attempt may have multiple exclusion reasons; counts must not be summed as disjoint categories. Total excluded: 47 attempts.

Reason (overlapping)	Attempts
Billing not reconciled	2
Driver/source binding conflict	32
No valid official evaluator	26
No valid prediction patch	25
Raw/evaluator disagreement	1
Raw evidence not measured/verified	25

Table 6: Supporting observations and archive audit counts by contract. Resolved counts include only admitted attempts. Source revisions and complete contract bindings remain in the machine inventory.

Contract	Archive	Eval.	Admit	Resolved
LITE26-RESUME	26	17	17	1
LITE30-FAILED-RETRY	5	2	2	1
LITE30-REMAINING-R10	28	18	0	0
LITE50-SERIAL-11001	1	0	0	0
LITE50-SERIAL-11001-R2	1	1	1	0
LITE50-SERIAL-11001-R3	1	1	1	1
LITE50-SERIAL-11019	1	1	1	0
LITE50-SERIAL-11019-R2	1	0	0	0
LITE50-SERIAL-11019-R3	1	1	1	0
LITE50-SERIAL-11019-R4	1	1	1	0
LITE50-SERIAL-11019-R5	1	1	0	0
LITE50-SERIAL-11019-R6	1	1	0	0
LITE50-SERIAL-11019-R7	1	1	0	0
LITE50-SERIAL-11019-R8	1	0	0	0
LITE50-SERIAL-11019-R9	1	0	0	0

The contract groups in Table 6 preserve the changing execution conditions. They do not define experimental treatment groups. Some exclusions occur before official evaluation; others concern evidence binding after a valid report. The study cannot infer model reasoning quality from a prediction rejection label alone, nor algorithm reliability from selected evaluator successes.

5.4. Evaluator-valid but evidence-inadmissible observations

Table 7 traces two retained attempts selected deterministically from evaluator-valid driver/source conflicts, using distinct task identities. In each case, the official evaluator completed an unresolved result and the raw runner record agrees. The declared driver digest nevertheless differs from the bytes at the declared source revision. The production core therefore excludes the observation from the admitted evidence set. Neither case is an evaluator false positive: the conflict concerns the identity of the evidence chain, not a mistaken resolved verdict.

Table 7: Real evidence-chain cases. Driver digests are displayed as prefixes for readability; full digests appear in Appendix B and the artifact. Actual driver means bytes at the declared source revision. Both attempts have valid, unresolved evaluator reports and are excluded for driver/source binding conflict.

Object	Observation A	Observation B
Task	django__django-11019	astropy__astropy-6938
Attempt	RUN-20261002T082314Z-D3E0297D	RUN-20261002T143647Z-4D06C2E5
Contract	EXP-CACTUS-SWEBENCH-LITE50-SERIAL-11019-R5-20261002T082226Z	EXP-CACTUS-SWEBENCH-LITE30-REMAINING-R10-20261002T142710Z
Source revision	96c4bde1a7ba77577c6c2498e9fd85367ad89089	4297948af2c42156b672d83f2a08ed4a320c35c5
Declared driver (prefix)	a1558c4d484253d4	22bfa3e9d4bf829e
Actual driver (prefix)	f4645ea7e49d190d	84810f0f277cfa78
Evaluator valid	true	true
Raw solved	re- false	false
Admitted	false	false

The frozen contracts and RUN evidence remain unchanged. Replacing their recorded digest retrospectively would create a different evidence claim without establishing what the original execution actually ran. The case artifact therefore retains both digests, the contract hash, bundle tree hash, raw result, and exclusion reason. Display prefixes are reading aids; identity verification uses the complete retained hashes. These illustrative cases explain the rule; the archive inventory supplies its prevalence count.

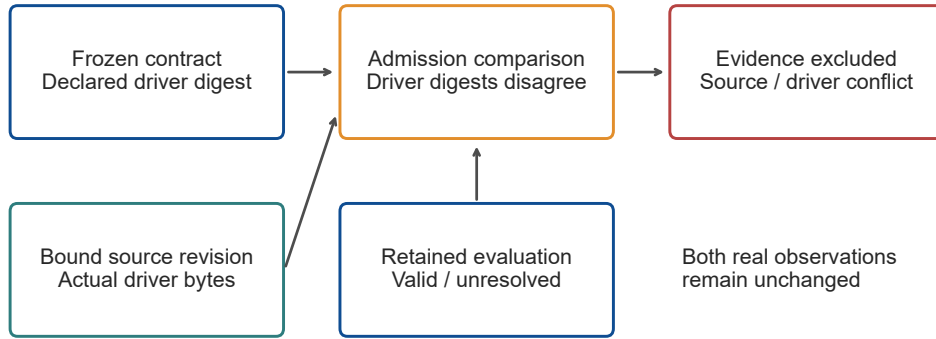


Figure 5: Observed evidence chain shared by the two cases in Table 7. A valid unresolved evaluation remains recorded, while disagreement between the declared driver digest and the bytes at the bound source revision prevents admission.

5.5. Repository archive integrity

The complete inventory covers 3848 directories, including 3788 directories named as RUNs. Under the current integrity and schema checks, 3788 directories are verified bundles and 60 are invalid, incomplete, or diagnostic directories without formal admission. Archive integrity remains failed. These operational counts describe retained evidence state, not the number of independent experiments or valid task outcomes.

5.6. Controlled semantic admission validation (RQ2)

All 160 resealed mutation cases passed integrity and were rejected at the semantic stage; the 20 controls were admitted. Each case also produced its specified reason code. Table 8 separates integrity passage from semantic rejection, and Table 9 derives its confusion matrix from individual admission

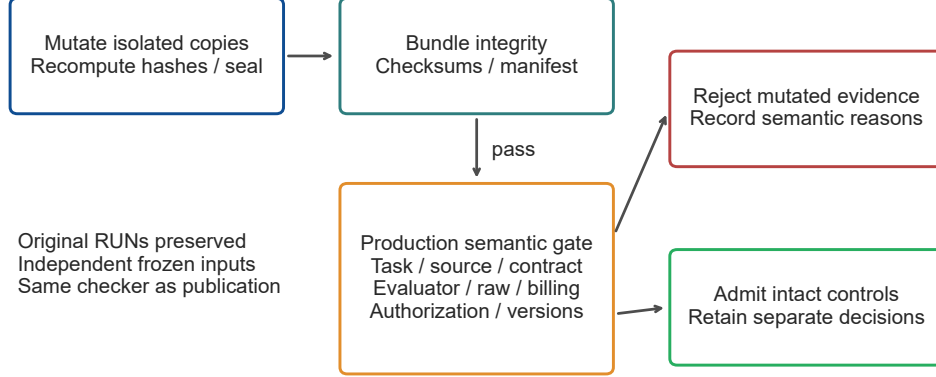


Figure 6: Semantic validation design. Isolated mutations are resealed before the production admission checker runs. The integrity layer passes before semantic rejection, while intact controls are admitted.

decisions. Both the production publication importer and this harness use the same core described in Section 3.3.

Table 8: Checksum-consistent semantic mutations. Intact means passing the entire bundle integrity layer before semantic admission. Every operator also reached its specified reason code.

Operator	Cases	Intact	Semantic	FN	Median (ms)
Cross-task prediction	20	20	20	0	31.49
Source revision	20	20	20	0	32.23
Driver digest / frozen contract	20	20	20	0	30.82
Raw/evaluator contradiction	20	20	20	0	31.15
Cross-attempt billing	20	20	20	0	31.43
Task identity	20	20	20	0	31.47
Authorization / contract	20	20	20	0	31.02
Evaluator image drift	20	20	20	0	32.25

All 20 of 20 seeds had all 8 semantic mutations correctly rejected after integrity passage and with their intended reason codes. All 20 of 20 seeds admitted both near-valid variants. Table 10 reports within-seed coverage; Appendix C gives the actual overlapping reason codes.

The source-revision and task-identity mutations can trigger multiple se-

Table 9: Production-core semantic validation. Positive means a mutation and detection means rejection. Exact intervals use the two-sided Clopper–Pearson construction as a binomial reference; reused seeds preclude an independent population interpretation. Latency covers the original controls and mutations in one warm process.

Measure	Value	95% exact reference interval
True positives / false negatives	160 / 0	—
False positives / true negatives	0 / 20	—
Tested mutation recall	100.0%	[97.72%, 100.00%]
Control specificity	100.0%	[83.16%, 100.00%]
Control false-positive rate	0.0%	[0.00%, 16.84%]
Integrity / semantic rejections	0 / 160	—
Mean validation latency (ms)	33.68	—
Median validation latency (ms)	31.49	—
Nearest-rank p95 latency (ms)	47.74	—

Table 10: Seed-level validation summary. Each row requires complete operator coverage within a seed; the same seeds are reused across rows and cases are not independent samples.

Seed-level outcome	Seeds
All semantic variants pass integrity and reach intended rejection	20/20
Both near-valid variants pass integrity and admission	20/20

mantic reasons. A driver-digest substitution is rejected by frozen-contract equality; it is not an isolated test of driver executable behavior. A billing substitution is internally reconciled yet conflicts with the target attempt’s retained usage. Thus these cases test cross-object consistency beyond check-sum validation, without claiming independent coverage of every semantic predicate.

The earlier suite retained 20 seeds, 200 mutations, and 20 controls. Most operators in that suite were rejected by integrity checks before its former test checker executed. Its unchanged outcomes remain historical integrity evidence. The new study does not overwrite those outcomes or imply that the earlier harness used the shared production implementation.

These finite-sample rates describe the specified operators and reused seeds. The reference intervals quantify binomial uncertainty under an independent-trial model, rather than establishing that model for this designed suite. Repeated seed copies and deliberately selected operators

prevent interpreting their bounds as coverage for unseen faults. The wider specificity interval also makes the limited intact-control count explicit. Latency excludes materialization, model calls, and official evaluator execution. The experiment does not estimate unseen-fault prevalence, cold-start performance, or evaluator-compute savings.

Near-valid semantic controls. A separate offline suite applied 2 admissible variations to the same 20 retained seeds, producing 40 copied controls. Prediction controls changed JSON field order and added unused metadata while preserving the task identity and exact patch text. Billing controls consistently changed synthetic token counts across the raw record and reconciliation fields, preserving the recorded spend. They test the declared predicates with counterfactual records; they are neither new invoices nor newly evaluated repair attempts. Frozen contracts, independent authorization records, original seeds, and the earlier semantic suite remained unchanged.

Table 11: Separate near-valid controls. These copied counterfactuals test allowed variation and are not pooled with the intact controls in Table 9.

Allowed variation	Cases	Integrity passed	Admitted
Prediction representation and metadata	20	20	20
Consistent synthetic usage variant	20	20	20

All 40 controls passed integrity and admission; 0 false positives were observed. Their descriptive exact specificity interval is [91.19%, 100.00%], and the false-positive interval is [0.00%, 8.81%]. Shared seeds and designed variants limit these reference intervals in the same way as the original suite. This check addresses over-rejection for the specified allowed variations, rather than every possible legal evidence transformation.

5.7. Secondary matched prediction-output archive (RQ3)

Endpoint and retained evidence. The archive contains 100 tasks with directories for each strategy. We recompute the binary endpoint directly from prediction files: a nonempty `model_patch` containing either `diff --git` or `---`. A missing prediction counts as output-format failure within this fully enumerated archive. This permissive text predicate does not establish syntactic validity, patch applicability, evaluator validity, or repair success. No official evaluator reports occur in these strategy directories. Patch length

is Python character count among outputs meeting the predicate, not token count.

Selection and configuration. The retained cohort targets SWE-bench Verified revision 78f471bf655a3137b2e8a75af1501690ec009ec3. Let P and F denote the counts of passing and failing benchmark tests. Tasks are sorted by $P/\max(1, P+F)$ and task identity, split into equal rank quartiles, and selected by deterministic round-robin repository traversal within each quartile. Equal numbers are taken from the quartiles. This test-count ratio is a sampling proxy, not an independently validated difficulty score. The recorded selection seed is unused by this deterministic selection code. Gold patches and test patches are absent from the inference manifest. Its declared digest matches LF-normalized text but not the retained CRLF bytes; our audit records the complete actual byte hash without rewriting historical files.

The runner requests the same code-context budget of 16384 for each strategy. Its referenced configuration identifies `deepseek-coding` through a private LiteLLM-compatible provider, with declared model version `DeepSeek V4 Flash (no-think)`, output cap 32768, initial temperature 0.0, and a maximum of two prediction attempts; retry temperature is 0.2. These are current configured values, not an independently established immutable provider model snapshot. The archive lacks a complete frozen per-treatment binding of prompt, model snapshot, driver, evaluator, and container identity. We therefore report paired archive associations rather than an isolated causal effect.

Results and statistics. Table 12 reports prediction-output rates and conditional patch lengths. Cochran’s paired binary test gives $Q = 15.51$, $df = 3$, and $p = 0.0014$. The tested endpoint is prediction output.

Table 12: Matched archive prediction-format outcomes. No evaluator-confirmed repair rates are reported. Patch lengths are character counts conditional on meeting the output predicate.

Strategy	Outputs / tasks	Output rate	Median length	IQR
FACO	37/100	37%	487.0	156.0
BM25	36/100	36%	508.5	157.2
STRUCTURAL	23/100	23%	483.0	119.5
MAXIMAL	20/100	20%	460.5	105.2

Pairwise exact two-sided McNemar probabilities are recomputed from discordant counts; Holm step-down adjustment covers the three FACO comparisons. Confidence intervals use paired task resampling, percentile bounds, 2000 resamples and seed 20261006. They are marginal intervals without simultaneous coverage adjustment. Differences and interval bounds are in percentage points.

Table 13: Paired prediction-output comparisons. Both: both strategies meet the output predicate; FO: FACO only; CO: control only; Neither: neither strategy meets it. CI: marginal bootstrap percentile interval.

Control	Both	FO	CO	Neither	Difference (95% CI; pp)	Exact p	Holm p
MAXIMAL	12	25	8	55	+17 pp [+7.00, +28.00]	0.0046	0.0091
STRUCTURAL	20	17	3	60	+14 pp [+6.00, +22.00]	0.0026	0.0077
BM25	20	17	16	47	+1 pp [-10.00, +12.02]	1.0000	1.0000

FACO has a higher output-format rate than MAXIMAL and STRUCTURAL within this archive after Holm adjustment; no difference is detected against BM25. This neither establishes statistical equivalence with BM25 nor supports a claim about repair correctness, attention dilution, or semantic density. A failure-profile distribution requires separate coding and is outside this RQ.

6. Discussion

The archive audit shows that evaluator validity and evidence admissibility diverge in practice. Among 71 archived attempts, 32 violate the driver/source binding. The two case chains show this conflict in attempts whose official evaluations completed normally. The resealed mutation study isolates cross-object inconsistencies: all 8 tested operator classes reach the semantic checker after bundle integrity passes.

These findings show why source identity and provenance need to be checked independently of the evaluator result. A completed evaluator run supplies a patch outcome; admission also requires agreement with the declared source and contract. Repository-level workflows often distribute predictions, contracts, source revisions, and evaluator reports across components. Checking their bindings at admission makes conflicts visible before aggregation, and the ledger carries those decisions into publication.

The matched prediction archive addresses a different question: how often each strategy forms nonempty, diff-marked output. Its inclusion illustrates why a strategy comparison must retain the meaning of its endpoint throughout analysis and reporting. The observed differences can guide investigation of output formation, while repair comparisons require official evaluator evidence.

Future validation should address the evidence gaps described in Section 7. Static checks need a replay study measuring missed detections, false positives, latency, and evaluator time. Task-level characterization needs a consistent terminal-task ledger, a frozen codebook, independent coding, and adjudication. Stronger strategy comparisons also require immutable provider and treatment bindings.

7. Threats to Validity

7.1. Construct validity

Prediction-format success is a permissive text predicate, distinct from valid patch application and repair success. Mutation rejection is distinct from correct attribution to a semantic rule. Task-level classification aggregates are withheld because they conflict with listed identities; the failure model is conceptual and not independently validated. Checks of retained image and model declarations do not prove the identity of a historical runtime or provider snapshot.

7.2. Internal validity

The matched output archive shares task identities but lacks complete immutable per-treatment model, prompt, driver, and container bindings. Retry behavior, provider nondeterminism, and changing implementation versions may confound observed strategy differences. Current configuration values do not establish historical execution settings. The semantic suite uses the production admission core and repeated seed copies. Its specified faults are known to the test designer, and multiple reasons can reject one mutation. Source reads may be cached within the process, so reported latency is not cold-start latency.

7.3. External validity

The prediction cohort uses deterministic test-count-ratio stratification and repository traversal, rather than random sampling or a validated difficulty metric. The retry archive is failure-selected and changed contracts

during development. Its admitted observations do not estimate a benchmark-wide repair rate. Synthetic fault operators cover only their specified mutations; observed rejection rates do not establish performance against unseen faults, attacks that preserve all checked predicates, or omitted evidence fields.

7.4. Conclusion validity

Paired tests describe the retained prediction-output endpoint. Holm correction covers the FACO pairwise family; bootstrap intervals are marginal. Repeated retry attempts and repeated mutation seeds are not independent task observations. Neither non-significance against BM25 nor a high mutation detection fraction implies equivalence, universal reliability, or repair superiority.

7.5. Reliability

Source artifacts, derivation scripts, and actual byte hashes are retained. LF-normalized and byte-level manifest hashes are distinguished. Independent blinded human coding, agreement statistics such as Cohen’s κ , disagreement counts, and adjudicated labels remain unavailable. We cannot replace a second human coder with automated relabeling or report an invented agreement coefficient. Independent coding would be required to upgrade the conceptual failure model to an empirically validated taxonomy, consistent with explicit methodological reporting (Ralph et al., 2020; Petersen et al., 2015). An immutable provider snapshot would be needed for a stronger strategy comparison.

8. Conclusion

CACTUS checks the agreement of prediction, evaluator, source, contract, authorization, and billing evidence before admitting an evaluation observation. The audit identifies 32 driver/source conflicts among 71 archived attempts, including cases with valid evaluator reports. The shared production core rejects all 160 checksum-consistent mutations and admits all 20 intact controls in the designed study. The publication ledger preserves these decisions when producing values, tables, figures, and the manuscript. Comparative repair effectiveness remains outside the evidence available in this study.

Data and Code Availability

The public review artifact contains the shared admission core, schemas, mutation operators and synthetic regression fixtures, a sanitized retry inventory and case-chain projection, archived citation metadata, publication source descriptors, value-ledger tooling, figure generators, and reproduction scripts. Its README distinguishes the following levels:

- **Paper regeneration:** Recompute tables, figures, and prose values from packaged, pinned source projections, then compile and check the PDF. This offline path checks the package’s hashes and rendering; it does not certify the omitted original RUN archive.
- **Admission verification:** Execute the shared core against synthetic regression fixtures and inspect the recorded semantic-study decisions. Recomputing admission of the original observed cases still requires their unredacted RUNs, source history, and frozen authorization records retained in the research workspace. Sanitized projections cannot replace those objects.
- **Full replay:** New model inference and official evaluation require benchmark licensing, model credentials, pinned runtime images, and separately authorized infrastructure. Full end-to-end replay is not asserted.

The package preserves full source digests, value identifiers, transformation rules, and declared data roles. Its release manifest identifies the included and omitted materials. The public review artifact is available from GitHub and is archived under exact version DOI 10.5281/zenodo.23226778. The artifact series has concept DOI 10.5281/zenodo.23205936. Anonymous downloads and the archived file identities were verified against the release checksums. The downloaded package reproduced its value ledger, tables, figures, and packaged manuscript baseline in a fresh virtual environment on the same host with locked dependencies. CACTUS-origin material is distributed under CC-BY-4.0; third-party materials retain their own notices. The original observed RUNs, source history, independent authorization records, and billing identities are not distributed in this artifact. Public availability and same-host regeneration do not certify original-archive re-admission, full model/evaluator replay, or independent external artifact evaluation. Anonymity is not asserted.

Appendix A. Context-selection and edit implementation

The following source-supported details are retained for implementation inspection. They specify current defaults, not experimentally optimized settings or isolated component improvements.

Appendix A.1. Task and context budget

A task supplies a repository, a base revision, an issue description, and an evaluator specification. Static indexing produces candidate units with paths, syntax spans, symbol identities, token counts, and source hashes. Let $\mathcal{C}$ be the candidate set, S the selected subset, and B the code-context budget. The selector enforces

$$\sum_{c \in S} \text{Tokens}(c) \leq B. \quad (\text{A.1})$$

Protected units and required parent definitions consume budget before optional units are added. If their complete closure exceeds the budget, selection returns a budget-infeasible status. The mechanism is a deterministic feasible heuristic; no optimality or approximation guarantee is claimed.

Appendix A.2. Static relevance and marginal utility

The current static scorer combines lexical relevance L , exact-symbol match M , graph proximity G , test membership T , and path match P . Its source-defined raw weights give

$$\begin{aligned} R(c) = & [0.30L(c) + 0.25M(c) \\ & + 0.20G(c) + 0.10T(c) \\ & + 0.10P(c)]/0.95. \end{aligned} \quad (\text{A.2})$$

The denominator is computed from the raw weights in the same source file. Static reproducible mode disables interactive path-read boosts. The described scorer does not require an embedding model.

The selector in `retrieval/marginal_utility.py` computes

$$\begin{aligned} \Delta(c \mid S) = & 1.00R(c) \\ & + 0.20 \text{CoverageGain}(c, S) \\ & - 0.15 \text{Redundancy}(c, S) \\ & - 0.15 \text{Fragmentation}(c). \end{aligned} \quad (\text{A.3})$$

Marginal density divides this utility by token cost plus a soft new-file charge. The default charge coefficient is 0.10. Coverage and redundancy are recomputed after a selection, and parent-closure cost is checked before admission. Defaults also specify 64 tokens for the fragmentation scale, 4 chunks per file, and a soft new-file threshold of 40. These are implementation parameters, not experimentally optimized values.

Deterministic tie-breaking uses score, path, source position, symbol identity, and unit identity. Context packaging retains unit source hashes and emits a canonical bundle hash. Recomputing marginal scores in the greedy loop means the implementation cannot be described as a single initial sort. Selection telemetry and matched ablations are required to measure its practical cost and benefit.

Appendix A.3. Source-owned edit sites

Relevant retrieval does not ensure applicable replacement text. CACTUS enumerates eligible syntax sites from the checked-out revision. A site binds a path, source span, old text, and digest. The model chooses listed identifiers and proposes replacement text. The renderer rejects changed source identity, duplicate or overlapping sites, and no-op edits. It reconstructs modified files, parses their syntax, and then emits a unified diff.

The current renderer permits 1–4 non-overlapping sites in an edit group. This limit is read from the renderer’s syntax tree rather than maintained independently in the manuscript. Site binding and parse checks establish local applicability; they do not establish a correct repair. Official evaluation remains necessary.

Appendix A.4. Dependent repair transactions

Supported edits can involve a helper and its callers. A transaction retains source-derived members, their roles and dependencies, and proposed replacements. Validation credentials bind the relevant contract and candidate contents. Changing a member invalidates its credential and dependent credentials, while retaining candidate text for rechecking. Incomplete transactions cannot commit. Validation of the required closure permits atomic patch emission.

Repeated unchanged failure obligations trigger a bounded stagnation stop and design review. The specialized semantic obligations presently cover supported sequence-ordering structures, including method binding and aggregation behavior. This is not a universal semantic checker. Neither local

validation nor atomic emission substitutes for the task evaluator. The mechanisms have not been isolated in matched component ablations.

Appendix B. Complete evidence-chain driver digests

Table B.14 retains the complete driver digests underlying the display prefixes in Table 7. Contract hashes and bundle tree hashes remain in the fixed case-chain artifact.

Table B.14: Complete SHA-256 driver identities for the retained case-chain observations. Identity verification uses these full values, rather than the main-text display prefixes.

Object	Complete SHA-256
A: declared driver	a1558c4d484253d4ef2a952d46b47f7f859e22aabc7fc3dbb747f6fb1a19a309
A: source driver	f4645ea7e49d190d252b08406a90c7072221b22fc81e634c03e909153face1be
B: declared driver	22bfa3e9d4bf829ebaae35a23568a550e9d66d75e021f487addaad2232fe40eb
B: source driver	84810f0f277cfa788724b9963a624334d9c4011d14923b2b61ba8543838264ec

Appendix C. Semantic reason coverage

The following matrix is generated directly from the pinned mutation decisions and the operator-to-intended-reason mapping. A case can trigger several codes; these overlapping reasons do not establish isolated coverage of each predicate. Counts describe retained cases and introduce no additional experiment or independent sample.

Table C.15: Mutation operator and semantic reason coverage. Additional reasons overlap; counts are cases triggering each code, not independent tests.

Operator	Intended reason	Additional reasons (cases)	Cases
Cross-task prediction	no_valid_prediction_patch	—	20
Source revision	source_commit_mismatch	contract_driver_source_hash_mismatch (20)	20
Driver digest / frozen contract	contract_frozen_mismatch	—	20

Operator	Intended reason	Additional reasons (cases)	Cases
Raw/evaluator contradiction	<code>raw_evaluator_result_mismatch</code>	—	20
Cross-attempt billing	<code>billing_attempt_usage_mismatch</code>	<code>billing_attempt_spend_mismatch</code> (20)	20
Task identity	<code>task_identity_mismatch</code>	<code>evaluator_image_contract_mismatch</code> (20) <code>no_valid_official_evaluator</code> (20) <code>no_valid_prediction_patch</code> (20) <code>raw_evaluator_result_mismatch</code> (20)	20
Authorization / contract	<code>authorization_contract_mismatch</code>	—	20
Evaluator image drift	<code>evaluator_image_contract_mismatch</code>	—	20

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

Research expenses were covered by internal research project resources at Taiyuan University of Technology. No third-party funding was received, and no project name or grant identifier applies. Apart from covering research expenses, the university had no role in study design, data analysis, or the decision to submit the manuscript.

CRedit authorship contribution statement

Zhao Daping: Conceptualization, Supervision. **Shi Honglei:** Conceptualization, Methodology, Software, Validation, Data curation, Formal analysis, Visualization, Writing – original draft, Writing – review & editing, Supervision.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT, OpenAI Codex, and Cursor AI-assisted features for coding assistance, testing, document typesetting, and compilation. OpenAI Codex also assisted with manuscript organization, language refinement, presentation review, and implementation of statistical, validation, and publication tooling. The figures were rendered from code; no figure was directly generated or edited by a generative image tool. The authors reviewed and edited the relevant AI-assisted text and code, verified the results, and take full responsibility for the content of the publication.

References

- Alur, R., Bodik, R., Juniwal, G., Martin, M.M.K., Raghothaman, M., Seshia, S.A., Singh, R., Solar-Lezama, A., Torlak, E., Udupa, A., 2013. Syntax-guided synthesis, in: 2013 Formal Methods in Computer-Aided Design, pp. 1–8. doi:10.1109/fmcad.2013.6679385.
- Association for Computing Machinery, 2020. Acm sigir artifact badging. ACM policy. URL: <https://sigir.org/general-information/acm-sigir-artifact-badging/>. version 1.1; accessed 2026-10-07.
- Badertdinov, I., Golubev, A., Nekrashevich, M., Shevtsov, A., Karasik, S., Andriushchenko, A., Trofimova, M., Litvintseva, D., Yangel, B., 2025. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv preprint arXiv:2505.20411. URL: <https://arxiv.org/abs/2505.20411>, doi:10.48550/arXiv.2505.20411. neurIPS 2025.
- Bairi, R., Sonwane, A., Kanade, A., C, V.D., Iyer, A., Parthasarathy, S., Rajamani, S., Ashok, B., Shet, S., 2023. Codeplan: Repository-level coding using llms and planning. arXiv preprint arXiv:2309.12499. URL: <https://arxiv.org/abs/2309.12499>, doi:10.48550/arXiv.2309.12499.
- Ceka, I., Mitchell, H., Pujar, S., Buratti, L., Ramji, S., Yang, J., Kaiser, G., Ray, B., 2026. Understanding automated program repair agents through the lens of traceability: An empirical study. Proceedings of the ACM on

Software Engineering 3, 4450–4473. URL: <https://api.crossref.org/works/10.1145/3832285>, doi:10.1145/3832285.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374. URL: <https://arxiv.org/abs/2107.03374>, doi:10.48550/arXiv.2107.03374.

Cheney, J., Missier, P., Moreau, L., 2013. Constraints of the prov data model. W3C Recommendation. URL: <https://www.w3.org/TR/2013/REC-prov-constraints-20130430/>.

Deng, X., Da, J., Pan, E., He, Y.Y., Ide, C., Garg, K., Lauffer, N., Park, A., Pasari, N., Rane, C., Sampath, K., Krishnan, M., Kundurthy, S., Hendryx, S., Wang, Z., Bharadwaj, V., Holm, J., Aluri, R., Zhang, C.B.C., Jacobson, N., Liu, B., Kenstler, B., 2025. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? arXiv preprint arXiv:2509.16941. URL: <https://arxiv.org/abs/2509.16941>, doi:10.48550/arXiv.2509.16941.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., Wang, H., 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 1–79. doi:10.1145/3695988.

Jia, Y., Harman, M., 2011. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37, 649–678. doi:10.1109/tse.2010.62.

Jimenez, C.E., Yang, J., Geng, J., 2024. Swe-bench lite. Benchmark dataset. URL: <https://www.swebench.com/lite.html>. accessed 2026-10-03.

- Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K., 2023. Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770. URL: <https://arxiv.org/abs/2310.06770>, doi:10.48550/arXiv.2310.06770.
- Just, R., Jalali, D., Ernst, M.D., 2014. Defects4j: a database of existing faults to enable controlled testing studies for java programs, in: Proceedings of the 2014 International Symposium on Software Testing and Analysis, pp. 437–440. doi:10.1145/2610384.2628055.
- Kim, D., Nam, J., Song, J., Kim, S., 2013. Automatic patch generation learned from human-written patches, in: 2013 35th International Conference on Software Engineering (ICSE), pp. 802–811. doi:10.1109/icse.2013.6606626.
- Le Goues, C., Holtschulte, N., Smith, E.K., Brun, Y., Devanbu, P., Forrest, S., Weimer, W., 2015. The manybugs and introclass benchmarks for automated repair of c programs. IEEE Transactions on Software Engineering 41, 1236–1256. doi:10.1109/tse.2015.2454513.
- Lebo, T., Sahoo, S., McGuinness, D., 2013. Prov-o: The prov ontology. W3C Recommendation. URL: <https://www.w3.org/TR/2013/REC-prov-o-20130430/>.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:2005.11401. URL: <https://arxiv.org/abs/2005.11401>, doi:10.48550/arXiv.2005.11401.
- Li, H., Zhu, L., Zhang, B., Feng, R., Wang, J., Pan, Y., Barr, E.T., Sarro, F., Chu, Z., Ye, H., 2026. Contextbench: A benchmark for context retrieval in coding agents. arXiv preprint arXiv:2602.05892. URL: <https://arxiv.org/abs/2602.05892>, doi:10.48550/arXiv.2602.05892.
- Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P., 2023a. Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172. URL: <https://arxiv.org/abs/2307.03172>, doi:10.48550/arXiv.2307.03172.

- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su, Y., Sun, H., Huang, M., Dong, Y., Tang, J., 2023b. Agentbench: Evaluating llms as agents. arXiv preprint arXiv:2308.03688. URL: <https://arxiv.org/abs/2308.03688>, doi:10.48550/arXiv.2308.03688.
- Lu, S., Duan, N., Han, H., Guo, D., Hwang, S.w., Svyatkovskiy, A., 2022. Reacc: A retrieval-augmented code completion framework. arXiv preprint arXiv:2203.07722. URL: <https://arxiv.org/abs/2203.07722>, doi:10.48550/arXiv.2203.07722.
- Ma, C., Zhang, J., Zhu, Z., Yang, C., Yang, Y., Jin, Y., Lan, Z., Kong, L., He, J., 2024. Agentboard: An analytical evaluation board of multi-turn llm agents. arXiv preprint arXiv:2401.13178. URL: <https://arxiv.org/abs/2401.13178>, doi:10.48550/arXiv.2401.13178.
- Malka, J., Zacchiroli, S., Zimmermann, T., 2026. Docker does not guarantee reproducibility. arXiv preprint arXiv:2601.12811. URL: <https://arxiv.org/abs/2601.12811>, doi:10.48550/arXiv.2601.12811.
- Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>. editors; published 30 April 2013. Accessed 2026-10-03.
- OliveiraJr, E., Madeiral, F., Santos, A.R., von Flach, C., Soares, S., 2024. A vision on open science for the evolution of software engineering research and practice. arXiv preprint arXiv:2405.12132. URL: <https://arxiv.org/abs/2405.12132>, doi:10.48550/arXiv.2405.12132.
- Open Container Initiative, 2024. Oci image format specification: Content descriptors. Image specification version 1.1.0. URL: <https://raw.githubusercontent.com/opencontainers/image-spec/v1.1.0/descriptor.md>.
- Petersen, K., Vakkalanka, S., Kuzniarz, L., 2015. Guidelines for conducting systematic mapping studies in software engineering: An update. Information and Software Technology 64, 1–18. doi:10.1016/j.infsof.2015.03.007.

- Qi, Z., Long, F., Achour, S., Rinard, M., 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems, in: *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, pp. 24–36. doi:10.1145/2771783.2771791.
- Qin, B., Xie, Y., 2026. Agent retrieval bench: Evaluating repository context retrieval for coding agents. *arXiv preprint arXiv:2607.24882*. URL: <https://arxiv.org/abs/2607.24882>, doi:10.48550/arXiv.2607.24882. accessed 2026-09-19.
- Ralph, P., Ali, N.b., Baltes, S., Bianculli, D., Diaz, J., Dittrich, Y., Ernst, N., Felderer, M., Feldt, R., Filieri, A., de França, B.B.N., Furia, C.A., Gay, G., Gold, N., Graziotin, D., He, P., Hoda, R., Juristo, N., Kitchenham, B., Lenarduzzi, V., Martínez, J., Melegati, J., Mendez, D., Menzies, T., Moller, J., Pfahl, D., Robbes, R., Russo, D., Saarimäki, N., Sarro, F., Taibi, D., Siegmund, J., Spinellis, D., Staron, M., Stol, K., Storey, M.A., Taibi, D., Tamburri, D., Torchiano, M., Treude, C., Turhan, B., Wang, X., Vegas, S., 2020. Empirical standards for software engineering research. *arXiv preprint arXiv:2010.03525*. URL: <https://arxiv.org/abs/2010.03525>, doi:10.48550/arXiv.2010.03525.
- Renzullo, J., Reiter, P., Weimer, W., Forrest, S., 2025. Automated program repair: Emerging trends pose and expose problems for benchmarks. *ACM Computing Surveys* 57, 1–18. URL: <https://api.crossref.org/works/10.1145/3704997>, doi:10.1145/3704997.
- Sandve, G.K., Nekrutenko, A., Taylor, J., Hovig, E., 2013. Ten simple rules for reproducible computational research. *PLoS Computational Biology* 9, e1003285. doi:10.1371/journal.pcbi.1003285.
- Siddiq, M.L., Islam-Gomes, A., Sekerak, N., Santos, J.C.S., 2025. Large language models for software engineering: A reproducibility crisis. *arXiv preprint arXiv:2512.00651*. URL: <https://arxiv.org/abs/2512.00651>, doi:10.48550/arXiv.2512.00651.
- Smith, E.K., Barr, E.T., Le Goues, C., Brun, Y., 2015. Is the cure worse than the disease? overfitting in automated program repair, in: *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, pp. 532–543. doi:10.1145/2786805.2786825.

- SWE-bench Team, 2024. Swe-bench verified. Benchmark dataset. URL: <https://www.swebench.com/verified.html>. accessed 2026-10-03.
- Wang, X., Li, B., Song, Y., Xu, F.F., Tang, X., Zhuge, M., Pan, J., Song, Y., Li, B., Singh, J., Tran, H.H., Li, F., Ma, R., Zheng, M., Qian, B., Shao, Y., Muennighoff, N., Zhang, Y., Hui, B., Lin, J., Brennan, R., Peng, H., Ji, H., Neubig, G., 2024. Openhands: An open platform for ai software developers as generalist agents. arXiv preprint arXiv:2407.16741. URL: <https://arxiv.org/abs/2407.16741>, doi:10.48550/arXiv.2407.16741.
- Wilkinson, M.D., Dumontier, M., Aalbersberg, I.J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.W., da Silva Santos, L.B., Bourne, P.E., Bouwman, J., Brookes, A.J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C.T., Finkers, R., Gonzalez-Beltran, A., Gray, A.J., Groth, P., Goble, C., Grethe, J.S., Heringa, J., 't Hoen, P.A., Hooft, R., Kuhn, T., Kok, R., Kok, J., Lusher, S.J., Martone, M.E., Mons, A., Packer, A.L., Persson, B., Rocca-Serra, P., Roos, M., van Schaik, R., Sansone, S.A., Schultes, E., Sengstag, T., Slater, T., Strawn, G., Swertz, M.A., Thompson, M., van der Lei, J., van Mulligen, E., Velterop, J., Waagmeester, A., Wittenburg, P., Wolstencroft, K., Zhao, J., Mons, B., 2016. The fair guiding principles for scientific data management and stewardship. *Scientific Data* 3. doi:10.1038/sdata.2016.18.
- Wilson, G., Bryan, J., Cranston, K., Kitze, J., Nederbragt, L., Teal, T.K., 2017. Good enough practices in scientific computing. *PLOS Computational Biology* 13, e1005510. doi:10.1371/journal.pcbi.1005510.
- Xia, C.S., Deng, Y., Dunn, S., Zhang, L., 2024. Agentless: Demystifying llm-based software engineering agents. arXiv preprint arXiv:2407.01489. URL: <https://arxiv.org/abs/2407.01489>, doi:10.48550/arXiv.2407.01489.
- Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024. Swe-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793. URL: <https://arxiv.org/abs/2405.15793>, doi:10.48550/arXiv.2405.15793.
- Yang, S., Chiang, W.L., Zheng, L., Gonzalez, J.E., Stoica, I., 2023. Rethinking benchmark and contamination for language models with rephrased

samples. arXiv preprint arXiv:2311.04850. URL: <https://arxiv.org/abs/2311.04850>, doi:10.48550/arXiv.2311.04850.

Zhang, F., Chen, B., Zhang, Y., Keung, J., Liu, J., Zan, D., Mao, Y., Lou, J.G., Chen, W., 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. arXiv preprint arXiv:2303.12570. URL: <https://arxiv.org/abs/2303.12570>, doi:10.48550/arXiv.2303.12570.

Zhang, Y., Ruan, H., Fan, Z., Roychoudhury, A., 2024. Autocoderover: Autonomous program improvement. arXiv preprint arXiv:2404.05427. URL: <https://arxiv.org/abs/2404.05427>, doi:10.48550/arXiv.2404.05427.

Zheng, Z., Ning, K., Wang, Y., Zhang, J., Zheng, D., Ye, M., Chen, J., 2023. A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv preprint arXiv:2311.10372. URL: <https://arxiv.org/abs/2311.10372>, doi:10.48550/arXiv.2311.10372.